ARHITECTURA SISTEMELOR DE CALCUL · ASC

Computer Systems Architecture

Workbook — Exercises by Chapter

Graded exercises (Basic · Intermediate · Challenge) for every chapter, from number bases to machine-code encoding. Every exercise is solved step by step, with the same number, in the companion manual **Theory & Solutions**.

x86 (IA-32), 32-bit protected mode · NASM syntax · toolchain NASM + ALINK + OllyDbg
13 chapters · 156 exercises

Year 1 · Computer Science (English) · UBB FMI Cluj-Napoca

Cuprins

Computer Systems Architecture — Workbook

Click any line to jump to that section. Page numbers match the page footers and the PDF viewer.

How to use this workbook 1

Notation 2

PART I — DATA AND THE MACHINE

Chapter 1 — Data Representation and Number Bases 3

- Exercise 1.1 — To decimal 3
- Exercise 1.2 — Decimal to binary and hexadecimal 3
- Exercise 1.3 — Binary and hexadecimal by nibbles 3
- Exercise 1.4 — Hex digits of the x86 sizes 3
- Exercise 1.5 — Octal and hexadecimal through bits 3
- Exercise 1.6 — Base 9 to base 7 3
- Exercise 1.7 — Strings as bytes 3
- Exercise 1.8 — Minimum number of bits 4
- Exercise 1.9 — Which constants are valid? 4
- Exercise 1.10 — How many values can n digits hold? 4
- Exercise 1.11 — Same number, different representations 4
- Exercise 1.12 — Character arithmetic in NASM 4

Chapter 2 — Signed and Unsigned Numbers, Two's Complement 5

- Exercise 2.1 — Both interpretations 5
- Exercise 2.2 — Negative numbers on a byte 5
- Exercise 2.3 — Three methods for two's complement 5
- Exercise 2.4 — Reading a register 5
- Exercise 2.5 — Negative numbers on a word and on a dword 5
- Exercise 2.6 — When a byte is not enough 5
- Exercise 2.7 — Sign and zero extension 5
- Exercise 2.8 — Minimum number of bits (signed) 5
- Exercise 2.9 — True or false: instruction classes 6
- Exercise 2.10 — One addition, two meanings 6
- Exercise 2.11 — Taking the low byte of a word 6
- Exercise 2.12 — Numbers equal to their own complement 6

Chapter 3 — x86 Architecture: Registers, Flags, Segments 7

- Exercise 3.1 — Which part of EAX changes? 7
- Exercise 3.2 — Parts of a register 7
- Exercise 3.3 — Registers with fixed roles 7
- Exercise 3.4 — Flags after 7Fh + 1 7
- Exercise 3.5 — Flags after 80h + 80h 7
- Exercise 3.6 — Trace EAX through writes to its parts 7
- Exercise 3.7 — inc versus add 7
- Exercise 3.8 — Two memory operands 8
- Exercise 3.9 — Parity and auxiliary carry 8
- Exercise 3.10 — A full flag trace 8
- Exercise 3.11 — CS:EIP and how EIP changes 8
- Exercise 3.12 — Flags of subtraction and comparison 8

Cuprins (continued)

PART II — PROGRAMMING IN NASM

Chapter 4 — Assembly Basics: Program Structure, Data Definition, Directives, Operators 9

Exercise 4.1 — Memory content of a data segment 9
Exercise 4.2 — Find the errors 9
Exercise 4.3 — Lengths with $ 9
Exercise 4.4 — Reading an element of an array 9
Exercise 4.5 — Operators 9
Exercise 4.6 — Little endian views 9
Exercise 4.7 — EQU in brackets 10
Exercise 4.8 — Another full data segment 10
Exercise 4.9 — Where is a size specifier needed? 10
Exercise 4.10 — Precedence and truncation 10
Exercise 4.11 — A complete program with lengths 10
Exercise 4.12 — Reading a memory dump 10

Chapter 5 — Arithmetic: ADD/SUB, MUL/DIV, Type Conversions, Expressions 11

Exercise 5.1 — (a − b) + (c − d), words, unsigned 11
Exercise 5.2 — a · b − c, mixed sizes, unsigned 11
Exercise 5.3 — Where does the product go? 11
Exercise 5.4 — A division that crashes 11
Exercise 5.5 — (a + b) / c, careful with the division 11
Exercise 5.6 — a / b + c · d, signed 11
Exercise 5.7 — Signs of quotient and remainder 11
Exercise 5.8 — A chain of conversions 11
Exercise 5.9 — Adding and subtracting qwords 12
Exercise 5.10 — A long signed expression 12
Exercise 5.11 — A long unsigned expression 12
Exercise 5.12 — Bug hunt: (b·c − a) / (a + 4) + d 12

Chapter 6 — Overflow: CF versus OF 13

Exercise 6.1 — 0BCh + 0DEh 13
Exercise 6.2 — 50 + 100 13
Exercise 6.3 — −50 − 100 13
Exercise 6.4 — 8000h − 1 on a word 13
Exercise 6.5 — 0FFh · 2 with mul 13
Exercise 6.6 — −100 · 3 with imul 14
Exercise 6.7 — 1000 / 3 with div 14
Exercise 6.8 — Operators, then a subtraction 14
Exercise 6.9 — Overflow on words 14
Exercise 6.10 — All four combinations, for add and for sub 14
Exercise 6.11 — div versus idiv on a negative dividend 14
Exercise 6.12 — A complete line-by-line analysis 14

Chapter 7 — Bitwise Instructions, Shifts and Rotates, Building Values from Bits 15

Exercise 7.1 — Shifts and rotates of 96h 15
Exercise 7.2 — Check a single bit 15
Exercise 7.3 — Clear and set groups of bits 15
Exercise 7.4 — and, or, xor, not on the same byte 15
Exercise 7.5 — shr versus sar 15
Exercise 7.6 — Rotating through the carry 15

Cuprins (continued)

Exercise 7.7 — Swapping parts with one instruction 15
Exercise 7.8 — Count the 1 bits 15
Exercise 7.9 — Build a dword from a byte and a word 16
Exercise 7.10 — Build a byte from a word 16
Exercise 7.11 — Multiply by 10 with shifts 16
Exercise 7.12 — Reverse the bits of a byte 16

PART III — MEMORY, CONTROL FLOW AND STRINGS

Chapter 8 — Addresses, Addressing Modes, Offset Computation 17
Exercise 8.1 — Classify the addressing modes 17
Exercise 8.2 — Which segment register? 17
Exercise 8.3 — Valid or invalid? 17
Exercise 8.4 — An element of a dword array 17
Exercise 8.5 — v+4 versus [v+4] 17
Exercise 8.6 — LEA as a calculator 17
Exercise 8.7 — Offsets and overlapping reads 17
Exercise 8.8 — Sum of an array, two ways 18
Exercise 8.9 — A matrix stored by rows 18
Exercise 8.10 — Pointer arithmetic 18
Exercise 8.11 — Base or index: how NASM decides 18
Exercise 8.12 — Walk an array of words backwards 18

Chapter 9 — Comparisons, Jumps, Loops (if / while / for in Assembly) 19
Exercise 9.1 — How many times? (1) 19
Exercise 9.2 — How many times? (2) 19
Exercise 9.3 — Which jumps are taken? 19
Exercise 9.4 — Minimum of two numbers 19
Exercise 9.5 — Maximum of a string of signed bytes 19
Exercise 9.6 — Count the even words 19
Exercise 9.7 — Sum of the elements greater than 10 19
Exercise 9.8 — Factorial with a while loop 19
Exercise 9.9 — loope: skip the leading zeros 20
Exercise 9.10 — A loop whose body is too long 20
Exercise 9.11 — Is the number prime? 20
Exercise 9.12 — Bubble sort 20

Chapter 10 — Strings and String Instructions 21
Exercise 10.1 — Trace the string registers 21
Exercise 10.2 — Copy a string 21
Exercise 10.3 — Uppercase copy 21
Exercise 10.4 — Compare two strings 21
Exercise 10.5 — Sum of the bytes of each dword 21
Exercise 10.6 — Keep only the positive elements 21
Exercise 10.7 — Concatenate low and high bytes 21
Exercise 10.8 — A byte as two hex characters 22
Exercise 10.9 — Reverse a string with string instructions 22
Exercise 10.10 — Split by parity 22
Exercise 10.11 — Find a character and its position 22
Exercise 10.12 — Remove the spaces in place 22

Cuprins (continued)

PART IV — PROCEDURES AND MODULES

Chapter 11 — The Stack, CALL/RET, cdecl, printf/scanf, Files 23

Exercise 11.1 — What does this sequence do? 23
Exercise 11.2 — Follow ESP 23
Exercise 11.3 — Address or value? 23
Exercise 11.4 — Joining and splitting through the stack 23
Exercise 11.5 — Read two numbers, print (a+b)·(a−b) 23
Exercise 11.6 — Read until 0, print the maximum 23
Exercise 11.7 — Write 1…10 to a file 23
Exercise 11.8 — A procedure with three parameters 24
Exercise 11.9 — Print a byte signed and unsigned 24
Exercise 11.10 — Count the digits in a file 24
Exercise 11.11 — A GCD procedure 24
Exercise 11.12 — Bug hunt in a printing loop 24

Chapter 12 — Multimodule Programming: ASM + ASM and ASM + C 25

Exercise 12.1 — global or extern? 25
Exercise 12.2 — Brackets or not? 25
Exercise 12.3 — Names and formats 25
Exercise 12.4 — Why save EBX but not ECX? 25
Exercise 12.5 — max in assembly, called from C 25
Exercise 12.6 — Sum of the digits of a C string 25
Exercise 12.7 — Use count_vowels 25
Exercise 12.8 — ASM + ASM with scanf and printf 26
Exercise 12.9 — Identify the pieces of a call 26
Exercise 12.10 — Sum of an int array 26
Exercise 12.11 — A variable shared by two modules 26
Exercise 12.12 — cdecl versus stdcall 26

PART V — INSIDE THE INSTRUCTION

Chapter 13 — Machine Instruction Representation (Internal Format, ModR/M, SIB, Prefixes) 27

Exercise 13.1 — Decode a ModR/M byte 27
Exercise 13.2 — Fields of a direct-address instruction 27
Exercise 13.3 — Why does [esp] need a SIB? 27
Exercise 13.4 — Implicit prefixes 27
Exercise 13.5 — The four prefix categories 27
Exercise 13.6 — Decode with a displacement 27
Exercise 13.7 — Decode with a SIB byte 27
Exercise 13.8 — Encode two instructions 28
Exercise 13.9 — Byte-sized moves 28
Exercise 13.10 — Instruction lengths 28
Exercise 13.11 — A 32-bit displacement 28
Exercise 13.12 — All four prefixes at once 28

How to use this workbook

This workbook holds the exercises; the companion manual **Theory & Solutions** holds the full theory of each chapter and a detailed solution for **every** exercise, under exactly the same number (Exercise N.M = Chapter N, exercise M).

Working method

1. Read the theory of the chapter in the Theory & Solutions manual (each chapter there starts with the theory).
2. Solve the exercises here **on paper first**, in order. They are graded:
   - **Basic** — one idea at a time; do all of them.
   - **Intermediate** — several ideas combined, short programs.
   - **Challenge** — complete programs, traps, bug hunting.
3. For every exercise that asks for code: assemble it with NASM, link it with ALINK and run it step by step in OllyDbg (F8). Look at the registers and at the memory window after every important instruction.
4. Compare with the solution only after you have an answer. Read the *Theory used* and *Step by step* parts even when your answer was right — they often show a shorter way or a trap.
5. Tick the box ☐ in front of the exercise when you can solve it without looking.

Conventions

- Architecture: x86 (IA-32), 32-bit programs, **NASM** syntax, toolchain **NASM + ALINK + OllyDbg**.
- When a solution shows only the data and code parts, the rest of the program is the standard skeleton from Chapter 4 (`bits 32`, `global start`, `extern exit`, `import exit msvcrt.dll`, and `push dword 0` / `call [exit]` at the end).
- "Signed" means two's complement. When nothing is said, values are unsigned.
- Numbers without a suffix are decimal.

HOW TO BUILD A PROGRAM

```
nasm -fobj prog.asm
alink -oPE -subsys console -entry start prog.obj
```

Then open `prog.exe` in OllyDbg and step through it with F8 (F7 to step into a `call`).

Notation

Notation	Meaning
`1011b`, `1011`	a number written in binary (suffix **b**); in tables 1 is blue and 0 is orange
`7Fh`, `0x7F`	a number written in hexadecimal (suffix **h** or prefix **0x**)
`(212)3`	the digits 212 read in base 3
`a`	a variable name = its **address** (offset)
`[a]`	the **value** stored in memory at address `a`
`byte [a]`, `word [a]`, `dword [a]`	the value at `a` read as 1, 2 or 4 bytes
`DX:AX`, `EDX:EAX`	a pair of registers holding one value: the left one is the **high** part
`AL`, `AH`, `AX`, `EAX`	the low byte, high byte of AX, low word, and the whole 32-bit register
`CF OF ZF SF PF AF`	Carry, Overflow, Zero, Sign, Parity, Auxiliary-carry flags
`DF`	Direction flag (string instructions)
`$`	the location counter (current offset in the segment)
signed / *unsigned*	the two interpretations of the same bits
`→`	"becomes" / "gives"
☐	tick the box when you have solved the exercise

PART I

Data and the Machine

Number bases, signed and unsigned values, the x86 registers and flags.

CHAPTER 1

Data Representation and Number Bases

Bits, bytes and the number bases a programmer uses every day: binary, hexadecimal, octal and decimal, the conversions between them, the x86 data sizes, NASM constants and ASCII characters. Theory: Theory & Solutions manual, Chapter 1.

Basic

☐ Exercise 1.1 — To decimal BASIC

Convert to decimal: `1011 0110b`, `7Fh`, `0FFFFh`, `100h`, `(212)3`.

☐ Exercise 1.2 — Decimal to binary and hexadecimal BASIC

Convert to binary and to hexadecimal: 200, 1000, 65, 4095.

☐ Exercise 1.3 — Binary and hexadecimal by nibbles BASIC

Write in binary: `3Ch`, `0B7h`, `1F0Ah`. Write in hexadecimal: `1101 0111b`, `1 0000 0001b`, `10 1011 1100b`.

☐ Exercise 1.4 — Hex digits of the x86 sizes BASIC

How many hex digits does a byte, a word, a dword and a qword have? Write the largest unsigned value of each size in hexadecimal and in decimal.

Intermediate

☐ Exercise 1.5 — Octal and hexadecimal through bits INTERMEDIATE

Convert `(357)8` to hexadecimal and `2Ah` to octal, directly through bits (without decimal). Check both results in decimal.

☐ Exercise 1.6 — Base 9 to base 7 INTERMEDIATE

Convert `(125)9` to base 7 and check the result. Then convert `(1011.101)2` and `(A.8)16` to decimal.

☐ Exercise 1.7 — Strings as bytes INTERMEDIATE

Write the bytes (in hex) that represent the string `'Hi5'` and the string `'AsC 9'`. What does `AL` contain after `mov al, 'g'` followed by `sub al, 'a'-'A'`?

☐ Exercise 1.8 — Minimum number of bits INTERMEDIATE

What is the minimum number of bits needed to write, unsigned, the values 255, 256, 300 and 1000? What is the smallest x86 data type that can hold each?

☐ Exercise 1.9 — Which constants are valid? INTERMEDIATE

For each line say whether NASM accepts it and, if yes, what the destination register contains (in hex):

```
mov ax, ABCDh
mov ax, 0ABCDh
mov al, 256
mov eax, 0x1F
mov bl, 'a'
mov cl, 1010b
mov dx, 0FFh
```

Challenge

☐ Exercise 1.10 — How many values can n digits hold? CHALLENGE

(a) What is the largest number written with 5 binary digits, with 3 hexadecimal digits and with 4 octal digits? (b) How many different values can a word hold? (c) Show that 3 hex digits and 12 bits describe exactly the same set of values.

☐ Exercise 1.11 — Same number, different representations CHALLENGE

Which of these denote the same number: `128`, `80h`, `1000 0000b`, `(200)8`, `(11112)3`, `(400)5`? Explain how you decide.

☐ Exercise 1.12 — Character arithmetic in NASM CHALLENGE

`AL` holds the code of a lowercase letter and `BL` holds the code of a decimal digit character. (a) Give two different instructions that turn `AL` into the matching uppercase letter. (b) Turn `BL` into the value of the digit. (c) For `AL = 'q'` and `BL = '8'` give both registers in hex after your instructions. (d) Why can `and al, 0DFh` also be used in (a)?

CHAPTER 2

Signed and Unsigned Numbers, Two's Complement

The same bits can mean two different numbers. This chapter explains the unsigned and the signed (two's complement) interpretation, the ranges of each size, and how to make a value wider without changing it. Theory: Theory & Solutions manual, Chapter 2.

Basic

☐ Exercise 2.1 — Both interpretations BASIC

Give the unsigned and the signed interpretation (base 10) of: `7Fh`, `80h`, `0FEh`, `9Ch` (bytes), `8000h`, `0FFF0h` (words).

☐ Exercise 2.2 — Negative numbers on a byte BASIC

Represent on a byte (in hexadecimal): −1, −128, −100, 100, −5.

☐ Exercise 2.3 — Three methods for two's complement BASIC

Compute the two's complement of `1001 0011b` and of `0101 1000b` with all three methods and say which number each pattern and each result represents (signed).

☐ Exercise 2.4 — Reading a register BASIC

`AL = 0C8h`. What is `AL` in signed and in unsigned base 10? And `AX = 0FF38h`?

Intermediate

☐ Exercise 2.5 — Negative numbers on a word and on a dword INTERMEDIATE

Represent on a word: −200, −1, −129. Represent on a dword: −1, −200.

☐ Exercise 2.6 — When a byte is not enough INTERMEDIATE

Can −200 be represented on a byte? What does NASM store for `db -200`? Are `db 200` and `db -56` the same bytes?

☐ Exercise 2.7 — Sign and zero extension INTERMEDIATE

Sign-extend and zero-extend `0A5h` to a word and to a dword. Do the same for `7Ch`. Give each result in hex and its value.

☐ Exercise 2.8 — Minimum number of bits (signed) INTERMEDIATE

What is the minimum number of bits needed to represent, **signed**, each of: −3, −8, −9, 8, 127, −128? Write −9 in binary on that many bits.

☐ Exercise 2.9 — True or false: instruction classes INTERMEDIATE

True or false? Explain each. (a) `mov` interprets its operands as signed. (b) `add` gives different result bits for signed and unsigned operands. (c) `mul` treats its operands as unsigned. (d) `cmp` decides by itself whether a comparison is signed. (e) `movsx` should be used to widen an unsigned byte.

Challenge

☐ Exercise 2.10 — One addition, two meanings CHALLENGE

Add the bytes `0C8h` and `64h`. Give the result byte, then interpret operands and result both as unsigned and as signed. Are both readings correct? Explain why the same `add` works for both.

☐ Exercise 2.11 — Taking the low byte of a word CHALLENGE

A word holds `FF6Dh`. Give both interpretations. Then only its low byte `6Dh` is kept. What are the values now? Explain when taking the low byte of a signed word keeps its value.

☐ Exercise 2.12 — Numbers equal to their own complement CHALLENGE

Compute the two's complement of `80h` and of `00h` on a byte. What do you notice? What does this say about the signed range?

CHAPTER 3

x86 Architecture: Registers, Flags, Segments

What is inside an IA-32 processor from the programmer's point of view: the two units, the general registers and their parts, the segment registers, EIP and the flags register. Theory: Theory & Solutions manual, Chapter 3.

Basic

☐ Exercise 3.1 — Which part of EAX changes? BASIC

Which bits of `EAX` does each instruction change: `mov ah, 5`, `mov ax, 5`, `mov al, 5`, `mov eax, 5`?

☐ Exercise 3.2 — Parts of a register BASIC

`EAX = 12345678h`. What are `AX`, `AH`, `AL`? Write instructions that put `1234h` into `BX`.

☐ Exercise 3.3 — Registers with fixed roles BASIC

Which register (or part) is used implicitly: (a) as the counter of `loop`; (b) as a variable shift count; (c) for the high half of a `mul` by a word; (d) as the source and destination index of string instructions; (e) as the top of the stack; (f) for the return value of a C function?

☐ Exercise 3.4 — Flags after 7Fh + 1 BASIC

Give CF, OF, ZF, SF, PF and AF after `mov al, 7Fh` / `add al, 1`.

Intermediate

☐ Exercise 3.5 — Flags after 80h + 80h INTERMEDIATE

Give AL and the flags CF, OF, ZF, SF, PF after `mov al, 80h` / `add al, 80h`.

☐ Exercise 3.6 — Trace EAX through writes to its parts INTERMEDIATE

Give `EAX` after each line:

```
mov eax, 0FFFFFFFFh
mov al, 0
mov ah, 12h
mov ax, 1
mov al, -1
```

☐ Exercise 3.7 — inc versus add INTERMEDIATE

Compare the two sequences. Give AL, CF and ZF at the end of each and explain the difference.

```
stc                 ; sequence A          stc             ; sequence B
mov al, 0FFh                              mov al, 0FFh
inc al                                    add al, 1
```

☐ Exercise 3.8 — Two memory operands INTERMEDIATE

Why is `add [a], [b]` invalid? Rewrite it for two byte variables `a` and `b`, keeping the result in `a`.

☐ Exercise 3.9 — Parity and auxiliary carry INTERMEDIATE

(a) Give PF for the results `10h`, `9Ah`, `0FFh`, `00h`, `1234h` (word). (b) Give AF for `0Fh + 01h`, `18h + 08h` and `22h + 11h`.

Challenge

☐ Exercise 3.10 — A full flag trace CHALLENGE

Give AL and the flags CF, OF, ZF, SF after each instruction:

```
mov al, 0F0h
add al, 20h
sub al, 11h
inc al
and al, 0Fh
```

☐ Exercise 3.11 — CS:EIP and how EIP changes CHALLENGE

(a) What does `CS:EIP` contain at run time? (b) Why is `mov eip, eax` rejected? (c) List three ways in which `EIP` changes during execution.

☐ Exercise 3.12 — Flags of subtraction and comparison CHALLENGE

Give the result (if any) and CF, ZF, SF, OF after: (a) `mov al, 0` / `sub al, 1`; (b) `mov al, 5` / `cmp al, 5`; (c) `mov al, 3` / `cmp al, 0FDh`.

PART II

Programming in NASM

Program structure, data definition, arithmetic, overflow and bit manipulation.

CHAPTER 4

Assembly Basics: Program Structure, Data Definition, Directives, Operators

How a NASM program is built: the skeleton, the elements the assembler works with, defining data in memory (little endian!), the location counter, EQU constants and the operators evaluated at assembly time. Theory: Theory & Solutions manual, Chapter 4.

Basic

☐ Exercise 4.1 — Memory content of a data segment BASIC

Write the memory content (hex, starting at offset 0) generated by the lines below, and the value of `lx`.

```
x db 1, -1, 'A'
y dw 1234h, -2
z dd 'ab', 100h
lx equ y - x
```

☐ Exercise 4.2 — Find the errors BASIC

What is wrong in each line? `a db [b]`, `b dd ebx`, `c dw times 2 5`, `d db 1000`.

☐ Exercise 4.3 — Lengths with $ BASIC

Given `s db 'abcdef'` followed by `n equ $-s`, what is `n`? What if `db 0` sits between them? What is `m` for `w dw 1, 2, 3` followed by `m equ ($-w)/2`?

☐ Exercise 4.4 — Reading an element of an array BASIC

`v dw 10, 20, 30`. Write the instruction that puts the third element into `AX`. What would `mov ax, [v+2]` give?

Intermediate

☐ Exercise 4.5 — Operators INTERMEDIATE

Compute (in hex): `0F0h & 3Ch`, `~0 & 0FFh`, `1 << 4 | 1`, `12 / 5`, `12 % 5`, `-7 // 2`, `0F0h ^ 0FFh`.

☐ Exercise 4.6 — Little endian views INTERMEDIATE

`a dd 11223344h`. What are `byte [a+1]`, `word [a+1]`, `word [a+2]` and `dword [a]`?

☐ Exercise 4.7 — EQU in brackets INTERMEDIATE

Why does `mov eax, [n]` not work when `n equ 5`? What is the correct instruction?

☐ Exercise 4.8 — Another full data segment INTERMEDIATE

Starting at offset 0, write the bytes generated and give the offset of every label:

```
a1 dw 'xy', 'z'
a2 times 3 db 0FFh
a3 dd a2 - a1
a4 db -1, 255, 1 << 7
a5 dw $ - a1
```

☐ Exercise 4.9 — Where is a size specifier needed? INTERMEDIATE

For each instruction, say whether it is valid as written; if not, correct it (`v` is a byte variable, `x` a dword variable, `c` a byte variable): `mov [v], 0` · `inc [v]` · `mov [v], al` · `push [x]` · `mul [c]` · `mov eax, [x]` · `add [x], 5`

Challenge

☐ Exercise 4.10 — Precedence and truncation CHALLENGE

Compute the bytes generated by `t db 5|6 + 7&8, 2+3*4<<1, -128<<1, ~0F0h, 300 % 256` and explain each.

☐ Exercise 4.11 — A complete program with lengths CHALLENGE

Write a complete program that defines: the string `s` = `'ASC 2026'` and its length `ls`; the word array `v` with the values 1…5 and its number of elements `nv`; a reserved area `r` of 10 dwords; and two dword variables `lens`, `cnt`. The code must store `ls` into `lens` and `nv` into `cnt`, then exit.

☐ Exercise 4.12 — Reading a memory dump CHALLENGE

The memory starting at label `x` contains the bytes `78 56 34 12 FE FF`. What are `dword [x]`, `word [x+4]` (signed and unsigned), `byte [x+1]` and `word [x+3]`?

CHAPTER 5

Arithmetic: ADD/SUB, MUL/DIV, Type Conversions, Expressions

The arithmetic instructions of x86, numbers wider than one register, multiplication and division with their implicit operands, widening conversions, and a reliable method for computing any expression with mixed sizes. Theory: Theory & Solutions manual, Chapter 5.

Basic

☐ Exercise 5.1 — (a − b) + (c − d), words, unsigned BASIC

Compute `(a − b) + (c − d)` for the unsigned words `a = 50`, `b = 20`, `c = 7`, `d = 2`. Write the program and give the result register.

☐ Exercise 5.2 — a · b − c, mixed sizes, unsigned BASIC

Compute `a · b − c` with `a`, `b` unsigned bytes and `c` an unsigned word (`a = 20`, `b = 15`, `c = 100`).

☐ Exercise 5.3 — Where does the product go? BASIC

Give the registers that hold the result, and their values in hex, after each sequence: (a) `mov al, 7` / `mov bl, 2` / `mul bl` (b) `mov ax, 1000` / `mov bx, 70` / `mul bx` (c) `mov eax, 100000` / `mov ebx, 100000` / `mul ebx`

☐ Exercise 5.4 — A division that crashes BASIC

What happens at `mov ax, 300` / `mov bl, 1` / `div bl`? Fix it.

Intermediate

☐ Exercise 5.5 — (a + b) / c, careful with the division INTERMEDIATE

Compute `(a + b) / c` with `a`, `b` unsigned words and `c` an unsigned byte (`a = 1000`, `b = 200`, `c = 7`). Give quotient and remainder.

☐ Exercise 5.6 — a / b + c · d, signed INTERMEDIATE

Compute `a / b + c · d` (signed) for `a = −100000` (dword), `b = 300` (word), `c = −4`, `d = 25` (bytes).

☐ Exercise 5.7 — Signs of quotient and remainder INTERMEDIATE

Give the quotient and the remainder of `idiv` for −7 / 2, 7 / −2 and −7 / −2 (bytes, dividend in AX).

☐ Exercise 5.8 — A chain of conversions INTERMEDIATE

`AL = 0F4h`. Give AX after `cbw`, then EAX after `cwde`, then EDX after `cdq`. What would `movzx ebx, al` give instead?

☐ Exercise 5.9 — Adding and subtracting qwords INTERMEDIATE

For the qwords `x = 1FFFFFFFFh` and `y = 0FFFFFFFFh` compute `s = x + y` and `t = x − y` with 32-bit registers.

Challenge

☐ Exercise 5.10 — A long signed expression CHALLENGE

Compute (signed) `x + (2 − a·b) / (a·3) − a + c` with `a` byte, `b` word, `c` dword, `x` qword. Use `a = −5`, `b = 300`, `c = −100000`, `x = −7000000000` and store the result in the qword `r`.

☐ Exercise 5.11 — A long unsigned expression CHALLENGE

Compute (unsigned) `x − (a·b + c) / (a + 1)` with `a` byte, `b` word, `c` dword, `x` qword, for `a = 200`, `b = 1000`, `c = 70000`, `x = 5000000000`. Store the result in the qword `r`.

☐ Exercise 5.12 — Bug hunt: (b·c − a) / (a + 4) + d CHALLENGE

The program below should compute, **signed**, `(b·c − a) / (a + 4) + d` in `EAX`, with `a` byte, `b` word, `c` byte, `d` dword (`a = −7`, `b = 300`, `c = −5`, `d = 1000`). Find every bug, explain it, and give the corrected program and the final value of `EAX`.

```
mov al, [c]
imul word [b]
push dx
push ax
pop ebx
mov al, a
cbw
cwde
sub ebx, eax
mov cl, [a]
add cl, 4
movzx ecx, cl
mov eax, ebx
cdq
div ecx
add eax, d
```

CHAPTER 6

Overflow: CF versus OF

What overflow means, why the processor needs two flags for it, quick rules to predict CF and OF for additions and subtractions, how multiplication and division behave, and a complete method for analysing a sequence of instructions. Theory: Theory & Solutions manual, Chapter 6.

Basic

☐ Exercise 6.1 — 0BCh + 0DEh BASIC

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 0BCh
add al, 0DEh
```

☐ Exercise 6.2 — 50 + 100 BASIC

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 50
add al, 100
```

☐ Exercise 6.3 — −50 − 100 BASIC

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, -50
sub al, 100
```

☐ Exercise 6.4 — 8000h − 1 on a word BASIC

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov ax, 8000h
sub ax, 1
```

Intermediate

☐ Exercise 6.5 — 0FFh · 2 with mul INTERMEDIATE

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 0FFh
mov bl, 2
mul bl
```

☐ Exercise 6.6 — −100 · 3 with imul INTERMEDIATE

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, -100
mov bl, 3
imul bl
```

☐ Exercise 6.7 — 1000 / 3 with div INTERMEDIATE

What happens at `mov ax, 1000` / `mov bl, 3` / `div bl`?

☐ Exercise 6.8 — Operators, then a subtraction INTERMEDIATE

Detail every line (hex, unsigned, signed) and give CF and OF after the last one:

```
mov ch, 0C0h | 0Ah
mov dl, 0A0h >> 2
sub dl, ch
```

☐ Exercise 6.9 — Overflow on words INTERMEDIATE

Give the result and CF, OF for (a) `mov ax, 7FFFh` / `add ax, 7FFFh`; (b) `mov bx, 0` / `sub bx, 1`.

Challenge

☐ Exercise 6.10 — All four combinations, for add and for sub CHALLENGE

Find byte operands that produce each of the four combinations (CF, OF) = (0,0), (0,1), (1,0), (1,1) — first with `add`, then with `sub`. Justify each.

☐ Exercise 6.11 — div versus idiv on a negative dividend CHALLENGE

`mov ax, -300` / `mov bl, 3`. What happens with `idiv bl`, and what with `div bl`? Explain.

☐ Exercise 6.12 — A complete line-by-line analysis CHALLENGE

Detail the effect of each line (hex, unsigned, signed, CF, OF) and explain every overflow:

```
mov al, 9Ch
add al, 0E4h
mov bl, 7Fh
sub bl, al
```

CHAPTER 7

Bitwise Instructions, Shifts and Rotates, Building Values from Bits

Working with individual bits: the logical instructions and masks, shifts and rotates, the difference between shift operators and shift instructions, and a step-by-step method for building a value out of bits of other values. Theory: Theory & Solutions manual, Chapter 7.

Basic

☐ Exercise 7.1 — Shifts and rotates of 96h BASIC

What do `AL` and `CF` contain after `mov al, 96h` followed by each of (separately): `shl al, 2`, `shr al, 3`, `sar al, 3`, `rol al, 4`?

☐ Exercise 7.2 — Check a single bit BASIC

Write instructions that jump to the label `set` if bit 5 of `BL` is 1.

☐ Exercise 7.3 — Clear and set groups of bits BASIC

Make bits 0–3 of `AX` zero and bits 12–15 one, leaving the other bits unchanged. Give AX for an initial value of `5A5Ah`.

☐ Exercise 7.4 — and, or, xor, not on the same byte BASIC

`AL = 0B6h`. Give AL after each instruction applied separately: `and al, 0Fh`, `or al, 0Fh`, `xor al, 0Fh`, `not al`, `xor al, al`. Which of them changes the flags?

Intermediate

☐ Exercise 7.5 — shr versus sar INTERMEDIATE

(a) Give AL after `shr al, 2` and after `sar al, 2` for `AL = 0F0h`, and interpret both results. (b) Compare `sar` by 1 with `idiv` by 2 for −7.

☐ Exercise 7.6 — Rotating through the carry INTERMEDIATE

Give AL and CF after `mov al, 81h` / `stc` / `rcl al, 1`, and then after a following `rcr al, 1`.

☐ Exercise 7.7 — Swapping parts with one instruction INTERMEDIATE

With one instruction each: swap the nibbles of `AL` (3Ch → C3h); swap `AH` and `AL`; swap the two words of `EAX`.

☐ Exercise 7.8 — Count the 1 bits INTERMEDIATE

Count how many bits of 1 are in the dword `x` (use a loop with `shr` and `adc`). Use `x = 0F0F0F0Fh` and `x = 80000001h`.

Challenge

☐ Exercise 7.9 — Build a dword from a byte and a word CHALLENGE

Given the byte A and the word B, compute the dword C:

- bits 0–7 of C are 1;
- bits 8–11 of C are bits 4–7 of A;
- bits 12–19 of C are bits 2–9 of B;
- bits 20–23 of C are bits 0–3 of A;
- bits 24–31 of C are the high byte of B.

Give C for `A = 0B7h`, `B = 0E2D5h`.

☐ Exercise 7.10 — Build a byte from a word CHALLENGE

Given the word A, compute the byte B:

- bits 0–2 of B are bits 5–7 of A;
- bits 3–5 of B are 1;
- bits 6–7 of B are the inverted bits 0–1 of A.

Give B for `A = 0B5C3h`.

☐ Exercise 7.11 — Multiply by 10 with shifts CHALLENGE

Compute `EAX = 10 · x` for the dword `x` without `mul` / `imul`, using x·10 = x·8 + x·2. Try it with x = 1234.

☐ Exercise 7.12 — Reverse the bits of a byte CHALLENGE

Reverse the order of the bits of the byte `x` (bit 0 ↔ bit 7, bit 1 ↔ bit 6, ...) using a loop with `shr` and `rcl`. Give the result for `x = 0B4h` and for `x = 01h`.

PART III

Memory, Control Flow and Strings

Addressing, comparisons and jumps, loops, string instructions.

CHAPTER 8

Addresses, Addressing Modes, Offset Computation

How an instruction designates a memory location: FAR and NEAR addresses, the offset formula base + index·scale + constant, the addressing modes, the implicit segment registers, pointer arithmetic and LEA. Theory: Theory & Solutions manual, Chapter 8.

Basic

☐ Exercise 8.1 — Classify the addressing modes BASIC

Classify the addressing mode of each operand: `mov [ebx+4], eax`, `add ecx, 10`, `sub eax, [v]`, `inc dword [v+esi*4]`.

☐ Exercise 8.2 — Which segment register? BASIC

Which segment register is used for `[ebp+8]`, `[ebx+esi]`, `[esi+ebp]`, `[ebp*2]`, `[esp+4]`?

☐ Exercise 8.3 — Valid or invalid? BASIC

Which of these are invalid, and why? `[eax+ebx+ecx]`, `[esp*2]`, `[ebx*3]`, `[ebx*5]`, `[a+b]` (`a`, `b` variables), `[ebx-4]`.

☐ Exercise 8.4 — An element of a dword array BASIC

`v dd 10, 20, 30, 40`. Put `v[2]` into `EAX` using `ESI = 2`.

Intermediate

☐ Exercise 8.5 — v+4 versus [v+4] INTERMEDIATE

For `v dd 10, 20, 30, 40`, what is the difference between `mov eax, v+4` and `mov eax, [v+4]`? What does each load?

☐ Exercise 8.6 — LEA as a calculator INTERMEDIATE

Compute `EAX` after `mov ebx, 10` / `lea eax, [ebx*8 + ebx + 1]`. Then write a single `lea` for each: EAX = EBX·3 + 7; EAX = ECX·9; EDX = EAX·4 + EBX.

☐ Exercise 8.7 — Offsets and overlapping reads INTERMEDIATE

The data segment starts at offset 0:

```
a db 1, 2, 3
b dw 4, 5
c dd 6
```

(a) What is the offset of `c`? (b) What offset does `[b+2]` designate? (c) What are `word [a+1]` and `dword [b-1]`?

☐ Exercise 8.8 — Sum of an array, two ways INTERMEDIATE

Compute the sum of the dword array `v dd 5, 10, 15, 20, 25` twice: once with `[v + esi*4]` and an index, once with a pointer register `EBX` that advances by 4.

Challenge

☐ Exercise 8.9 — A matrix stored by rows CHALLENGE

A 3×4 matrix of bytes is stored row by row in `m`. Write the address formula of element (i, j) and load `m[2][1]` into `AL` with `i` in `ESI` and `j` in `EDI`.

☐ Exercise 8.10 — Pointer arithmetic CHALLENGE

With `a` and `b` variables in the data segment, which of the following are valid at assembly time, and is each a pointer or a scalar? `a+4`, `b-a`, `a+b`, `a*2`, `$-a`, `!a`, `~(b-a)`, `a-4`.

☐ Exercise 8.11 — Base or index: how NASM decides CHALLENGE

Which register is the base, and which segment register is used, for `[ebx+ebp]`, `[ebp+ebx]`, `[ebx+ebp*1]`, `[ebx*1+ebp*1]`, `[ebp*1+ebx*1]`? What does `mov eax, [es:ebx]` change?

☐ Exercise 8.12 — Walk an array of words backwards CHALLENGE

Copy the word array `s dw 1, 2, 3, 4, 5` into `d` in reverse order (d = 5, 4, 3, 2, 1), using one register as an index going up and another going down.

CHAPTER 9

Comparisons, Jumps, Loops (if / while / for in Assembly)

How decisions and repetition are written in assembly: CMP and the conditional jumps, signed versus unsigned conditions, jump distances, the LOOP family, and the translation of if, while, for and do-while. Theory: Theory & Solutions manual, Chapter 9.

Basic

☐ Exercise 9.1 — How many times? (1) BASIC

How many times does the body run? `mov ecx, 5` / `L: dec ecx` / `loop L`.

☐ Exercise 9.2 — How many times? (2) BASIC

How many times does the body run? `mov ecx, 0` / `L: inc ebx` / `loop L`.

☐ Exercise 9.3 — Which jumps are taken? BASIC

After `mov al, 80h` / `cmp al, 7Fh`, which of `ja`, `jg`, `jb`, `jl` jump?

☐ Exercise 9.4 — Minimum of two numbers BASIC

Store in `m` the minimum of the dwords `a = -5` and `b = 3`, first treating them as signed, then as unsigned. Why are the results different?

Intermediate

☐ Exercise 9.5 — Maximum of a string of signed bytes INTERMEDIATE

Compute the maximum of the string of signed bytes `s db 5, -7, 20, -100, 13` into `max`.

☐ Exercise 9.6 — Count the even words INTERMEDIATE

Count the even numbers in the string of words `w dw 3, 8, 10, 7, 2` into the byte `cnt`.

☐ Exercise 9.7 — Sum of the elements greater than 10 INTERMEDIATE

For the unsigned bytes `v db 4, 12, 200, 10, 11, 9`, compute in the word `s` the sum of the elements strictly greater than 10.

☐ Exercise 9.8 — Factorial with a while loop INTERMEDIATE

Compute `f = n!` for the byte `n = 6` into the dword `f`, using a while loop that stops when the counter reaches 1.

☐ Exercise 9.9 — loope: skip the leading zeros INTERMEDIATE

For `s db 0, 0, 0, 7, 0, 5`, use `loope` to find the index of the first non-zero byte (store it in `pos`, or `len` if all bytes are zero).

Challenge

☐ Exercise 9.10 — A loop whose body is too long CHALLENGE

A `loop L` gives *short jump is out of range* because its body is longer than 127 bytes. Rewrite it so that it still uses `loop` (and therefore changes no flags). Show that it works with a body of 200 `nop`s that also increments `EAX`, executed 3 times.

☐ Exercise 9.11 — Is the number prime? CHALLENGE

Set the byte `prime` to 1 if the dword `n` (n ≥ 2) is prime, else to 0, by trying all divisors d = 2, 3, … while d·d ≤ n. Check with n = 97 and n = 91.

☐ Exercise 9.12 — Bubble sort CHALLENGE

Sort the string of signed bytes `s db 5, -3, 12, 0, -8, 7` in ascending order with bubble sort (repeat passes until a pass makes no swap).

CHAPTER 10

Strings and String Instructions

The instructions that process memory sequentially — LODS, STOS, MOVS, SCAS, CMPS — their implicit operands ESI, EDI and DF, the repeat prefixes, XLAT, and the classic string patterns. Theory: Theory & Solutions manual, Chapter 10.

Basic

☐ Exercise 10.1 — Trace the string registers BASIC

`s dw 1234h, 5678h, 9ABCh` and `d` is a byte buffer. Give AX (or AL), ESI and EDI (as s+k, d+k) after each line:

```
mov esi, s
mov edi, d
cld
lodsw
stosb
lodsb
std
stosb
```

☐ Exercise 10.2 — Copy a string BASIC

Copy the string `s db 'Hello, NASM!'` into `d` with one repeated instruction.

☐ Exercise 10.3 — Uppercase copy BASIC

Build `d` = the uppercase version of `s db 'hello world'` (non-letters unchanged).

Intermediate

☐ Exercise 10.4 — Compare two strings INTERMEDIATE

Compare two strings `s1`, `s2` of equal length with `repe cmpsb` and set `eq = 1` if they are equal, 0 otherwise. Try `'abcde'` / `'abcde'` and `'abcde'` / `'abXde'`.

☐ Exercise 10.5 — Sum of the bytes of each dword INTERMEDIATE

Given a string of dwords, build a string of bytes where each byte is the sum (mod 256) of the 4 bytes of the matching dword. Use `s dd 01020304h, 0FFh`.

☐ Exercise 10.6 — Keep only the positive elements INTERMEDIATE

From `s db 3, -2, 0, 7, -9, 12` build `d` with only the strictly positive elements (signed), and count them in `n`.

☐ Exercise 10.7 — Concatenate low and high bytes INTERMEDIATE

Build `r` = the low bytes of the words of `s` followed by the high bytes of the words of `t`, for `s dw 1122h, 3344h` and `t dw 5566h, 7788h, 99AAh`.

☐ Exercise 10.8 — A byte as two hex characters INTERMEDIATE

Convert the byte `val db 0B7h` into its 2-character hexadecimal text `hx` using `xlat`.

☐ Exercise 10.9 — Reverse a string with string instructions INTERMEDIATE

Reverse `s db 'stack'` into `r` (r = 'kcats') by reading forward with `lodsb` and writing backward with `stosb`.

Challenge

☐ Exercise 10.10 — Split by parity CHALLENGE

Split `s db 4, 7, 10, 3, 8, 1, 6` into `ev` (even elements) and `od` (odd elements), keeping the order, and store their counts in `ne` and `no`.

☐ Exercise 10.11 — Find a character and its position CHALLENGE

Find the position (from 0) of the first `'m'` in `s db 'assembly language'` with `repne scasb`; store −1 in `pos` if there is none. Then explain the values of EDI and ECX after the search.

☐ Exercise 10.12 — Remove the spaces in place CHALLENGE

Remove all spaces from the 0-terminated string `t db 'a b c d', 0` **in place** (the result must also end with 0).

PART IV

Procedures and Modules

The stack, CALL/RET, the C library, files, multimodule programs.

CHAPTER 11

The Stack, CALL/RET, cdecl, printf/scanf, Files

How the stack works, how procedures are called and how they return, the cdecl calling convention of C, and how an assembly program uses printf, scanf and the C file functions. Theory: Theory & Solutions manual, Chapter 11.

Basic

☐ Exercise 11.1 — What does this sequence do? BASIC

`push eax` / `push ebx` / `pop eax` / `pop ebx` — what happened? Check with EAX = 1, EBX = 2.

☐ Exercise 11.2 — Follow ESP BASIC

Initially `ESP = 0019FF74h`. Give ESP after each line: `push eax`, `push ax`, `pop bx`, `push dword 7`, `pop ecx`, `pop edx`.

☐ Exercise 11.3 — Address or value? BASIC

Why do we write `push dword msg` for a text but `push dword [n]` for a number in `printf`? And why `push dword n` in `scanf`?

☐ Exercise 11.4 — Joining and splitting through the stack BASIC

Put the word result of `mul` (DX:AX) into EAX using the stack, then split `EBX = 11223344h` into `CX` (high word) and `SI` (low word).

Intermediate

☐ Exercise 11.5 — Read two numbers, print (a+b)·(a−b) INTERMEDIATE

Read two numbers `a` and `b` from the keyboard and print `(a+b)*(a-b)`. What is printed for the input `7 3`?

☐ Exercise 11.6 — Read until 0, print the maximum INTERMEDIATE

Read numbers until 0 is read; then print the maximum of the numbers read (0 excluded). What is printed for `4 -2 17 9 0`?

☐ Exercise 11.7 — Write 1…10 to a file INTERMEDIATE

Write the numbers 1…10, one per line, to the file `out.txt`.

☐ Exercise 11.8 — A procedure with three parameters INTERMEDIATE

Write a procedure `sum3(a, b, c)` with a stack frame that returns a + b + c in EAX, preserves EBX, and call it with 10, −4, 25 (cdecl).

☐ Exercise 11.9 — Print a byte signed and unsigned INTERMEDIATE

Print the byte `b db -5` once as a signed number and once as an unsigned number, on one line: `-5 251`.

Challenge

☐ Exercise 11.10 — Count the digits in a file CHALLENGE

Count the decimal digit characters in the file `data.txt` (of any length) by reading it in chunks of 8 bytes with `fread`, and print the count. For a file containing `Room 101, floor 3, code 7741`, what is printed?

☐ Exercise 11.11 — A GCD procedure CHALLENGE

Write a procedure `gcd(a, b)` (Euclid's algorithm with remainders) that returns the greatest common divisor in EAX, and use it from `start` to print `gcd(84, 36) = 12`.

☐ Exercise 11.12 — Bug hunt in a printing loop CHALLENGE

The program below should print the squares of 1…5, one per line, but it misbehaves. Find the bugs and give a corrected version.

```
segment data use32 class=data
    fmt db "%d", 10
segment code use32 class=code
start:
    mov ecx, 5
    mov ebx, 1
next:
    mov eax, ebx
    mul ebx
    push dword fmt
    push eax
    call [printf]
    inc ebx
    loop next
    push dword 0
    call [exit]
```

CHAPTER 12

Multimodule Programming: ASM + ASM and ASM + C

Splitting a program into separately assembled modules: global and extern, linking several object files, calling assembly functions from C, and the call, entry and exit code that make a function call work. Theory: Theory & Solutions manual, Chapter 12.

Basic

☐ Exercise 12.1 — global or extern? BASIC

`main.asm` calls the function `sum` and reads the variable `limit`; both are defined in `lib.asm`. Which declarations are needed in each module?

☐ Exercise 12.2 — Brackets or not? BASIC

Explain the difference between `call sum` and `call [printf]`. What would `call printf` (without brackets) do?

☐ Exercise 12.3 — Names and formats BASIC

A C program calls `int maxim(int a, int b)` written in assembly. (a) What label must the assembly function have? (b) What NASM format must be used? (c) Give the commands to build with MinGW.

☐ Exercise 12.4 — Why save EBX but not ECX? BASIC

Why must `_count_vowels` (section 12.4) save EBX but not ECX?

Intermediate

☐ Exercise 12.5 — max in assembly, called from C INTERMEDIATE

Write an assembly module with `int max(int a, int b)` (signed) used from a C `main` that reads two numbers and prints the maximum.

☐ Exercise 12.6 — Sum of the digits of a C string INTERMEDIATE

Write `int sum_digits(const char* s)` in assembly: the sum of the digit characters of a 0-terminated string. What does `sum_digits("a1b2c3")` return?

☐ Exercise 12.7 — Use count_vowels INTERMEDIATE

Using `_count_vowels` from section 12.4, what are `count_vowels("Assembly Is Fun")`, `count_vowels("xyz")` and `count_vowels("")`? Which registers does the function modify, and which does it restore?

☐ Exercise 12.8 — ASM + ASM with scanf and printf INTERMEDIATE

Write `main.asm` that reads two numbers, calls `max` from another assembly module and prints `max = …`. Give the module and the build commands. What is printed for `-4 -1`?

☐ Exercise 12.9 — Identify the pieces of a call INTERMEDIATE

In the code below, mark the call code, the entry code and the exit code, and say who would write each piece if the caller were C.

```
    push ecx
    push dword [n]
    push dword v
    call _f
    add esp, 8
    pop ecx
    ...
_f: push ebp
    mov ebp, esp
    sub esp, 4
    push esi
    ...
    pop esi
    mov esp, ebp
    pop ebp
    ret
```

Challenge

☐ Exercise 12.10 — Sum of an int array CHALLENGE

Write `int sum_array(const int* v, int n)` in assembly with a stack frame, using ESI as the pointer (so it must be saved). What does it return for v = {5, −2, 10, 7} and n = 4? And for n = 0?

☐ Exercise 12.11 — A variable shared by two modules CHALLENGE

Module `counter.asm` defines `count dd 0` and a procedure `bump` that adds 1 to it. `main.asm` calls `bump` three times and then prints `count`. Write both modules.

☐ Exercise 12.12 — cdecl versus stdcall CHALLENGE

Rewrite `sum(a, b)` so that the **callee** removes the parameters (stdcall style), and show how the call changes. Why does `printf` have to be cdecl?

PART V

Inside the Instruction

How an instruction becomes machine-code bytes.

CHAPTER 13

Machine Instruction Representation (Internal Format, ModR/M, SIB, Prefixes)

How the assembler turns an instruction into bytes: the internal format of an x86 instruction, the ModR/M and SIB bytes, displacements and immediates, and the four categories of prefixes. Theory: Theory & Solutions manual, Chapter 13.

Basic

☐ Exercise 13.1 — Decode a ModR/M byte BASIC

Decode the ModR/M byte `D8h` in the instruction `89 D8` (89 = MOV r/m32, r32).

☐ Exercise 13.2 — Fields of a direct-address instruction BASIC

Which fields appear in the encoding of `mov ecx, [v]` (direct addressing)? What changes for `mov eax, [v]`?

☐ Exercise 13.3 — Why does [esp] need a SIB? BASIC

Why does `mov ecx, [esp]` need a SIB byte although there is no index? Give its bytes.

☐ Exercise 13.4 — Implicit prefixes BASIC

Which prefix(es) does NASM generate (in 32-bit code) for `cwd`, `cdq`, `mov ax, [esi]`, `es lodsb`? Give the bytes.

Intermediate

☐ Exercise 13.5 — The four prefix categories INTERMEDIATE

Name the four categories of prefixes, say which ones the programmer writes explicitly, and give one example of each with its bytes.

☐ Exercise 13.6 — Decode with a displacement INTERMEDIATE

Decode `8B 4B 08` (8B = MOV r32, r/m32).

☐ Exercise 13.7 — Decode with a SIB byte INTERMEDIATE

Decode `8B 44 B3 10`.

☐ Exercise 13.8 — Encode two instructions INTERMEDIATE

Encode `mov edx, [eax]` (opcode 8B = MOV r32, r/m32) and `add ecx, edx` (opcode 01 = ADD r/m32, r32).

☐ Exercise 13.9 — Byte-sized moves INTERMEDIATE

Given the opcodes 88 = MOV r/m8, r8 and 8A = MOV r8, r/m8, encode `mov [ebx], al` and `mov al, [ebx]`.

Challenge

☐ Exercise 13.10 — Instruction lengths CHALLENGE

How many bytes does each instruction take, and which fields does it have? `mov eax, 12345678h`, `mov ecx, [ebx+esi*4+100h]`, `add byte [ebx], 5`, `mov ax, bx`, `rep movsb`.

☐ Exercise 13.11 — A 32-bit displacement CHALLENGE

Decode `8B 8B 00 01 00 00`.

☐ Exercise 13.12 — All four prefixes at once CHALLENGE

Explain every byte of `F3 2E 66 67 FF 33`, the encoding of `rep push dword [cs:ebx]` in `bits 16` code. Why do the 66 and 67 prefixes appear here, although they would not in 32-bit code?